

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5117

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5 , BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

*I Harshini R N (192421175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Harshini R N

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Write pseudocode to simulate a simplified TLS handshake process, including client request, server authentication using a digital certificate, session key generation, and secure data exchange.
2. Develop pseudocode for a firewall rule engine that filters incoming packets based on IP address, port number, and protocol, and decides whether to allow or block traffic.
3. Write pseudocode to detect and classify malware behavior (virus, worm, or Trojan) based on propagation method, file modification patterns, and user interaction.
4. Design pseudocode for an Intrusion Detection System (IDS) that uses both signature-based and anomaly-based detection to identify suspicious network activity.
5. Create pseudocode for an Intrusion Prevention System (IPS) that monitors network traffic in real time and automatically blocks malicious requests while logging intrusion attempts.

ANSWERS:

1. Simplified TLS Handshake Process

BEGIN TLS_HANDSHAKE

CLIENT sends "ClientHello" with supported ciphers and random value

SERVER receives "ClientHello"

SERVER sends "ServerHello" with chosen cipher, random value, and digital certificate

CLIENT verifies SERVER certificate with trusted CA

IF certificate valid THEN

CLIENT and SERVER perform key exchange (e.g., Diffie-Hellman)

SESSION_KEY ← derived from shared secret + random values

```
CLIENT sends "Finished" encrypted with SESSION_KEY  
SERVER sends "Finished" encrypted with SESSION_KEY  
SECURE_CHANNEL established  
ELSE  
    TERMINATE connection  
ENDIF  
SECURE DATA EXCHANGE using SESSION_KEY  
END TLS_HANDSHAKE
```

2. Firewall Rule Engine

```
BEGIN FIREWALL_ENGINE(packet)  
  
    INPUT: packet with source_IP, dest_port, protocol  
    RULES ← predefined list of allow/block conditions  
  
    FOR each rule IN RULES DO  
        IF packet.source_IP = rule.IP AND  
           packet.dest_port = rule.port AND  
           packet.protocol = rule.protocol THEN  
            IF rule.action = "ALLOW" THEN  
                RETURN "ALLOW"  
            ELSE  
                RETURN "BLOCK"
```

```
        ENDIF
    ENDIF
ENDFOR

RETURN "BLOCK" // default deny
END FIREWALL_ENGINE
```

3. Malware Behavior Detection

```
BEGIN MALWARE_DETECTION(activity)

    INPUT: activity with propagation_method, file_modification, user_interaction

    IF propagation_method = "requires host file" AND file_modification = "infects files"
    THEN

        CLASSIFY AS "VIRUS"

    ELSE IF propagation_method = "self-replicates via network" AND user_interaction =
"none" THEN

        CLASSIFY AS "WORM"

    ELSE IF propagation_method = "disguised software" AND user_interaction =
"installation required" THEN

        CLASSIFY AS "TROJAN"

    ELSE

        CLASSIFY AS "UNKNOWN"
```

ENDIF

RETURN classification

END MALWARE_DETECTION

4. Intrusion Detection System (IDS)

BEGIN IDS_MONITOR(network_traffic)

SIGNATURE_DB ← known attack patterns

NORMAL_PROFILE ← baseline of normal traffic behavior

FOR each packet IN network_traffic DO

IF packet matches SIGNATURE_DB THEN

ALERT "Signature-based intrusion detected"

ELSE IF packet deviates significantly from NORMAL_PROFILE THEN

ALERT "Anomaly-based intrusion detected"

ELSE

CONTINUE monitoring

ENDIF

ENDFOR

END IDS_MONITOR

5. Intrusion Prevention System (IPS)

```
BEGIN IPS_MONITOR(network_traffic)
```

```
    SIGNATURE_DB ← known malicious patterns
```

```
    ANOMALY_PROFILE ← baseline of normal traffic
```

```
    FOR each packet IN network_traffic DO
```

```
        IF packet matches SIGNATURE_DB OR deviates from ANOMALY_PROFILE
```

```
    THEN
```

```
        BLOCK(packet)
```

```
        LOG "Intrusion attempt blocked: " + packet.details
```

```
    ELSE
```

```
        FORWARD(packet) // allow normal traffic
```

```
    ENDIF
```

```
    ENDFOR
```

```
END IPS_MONITOR
```

